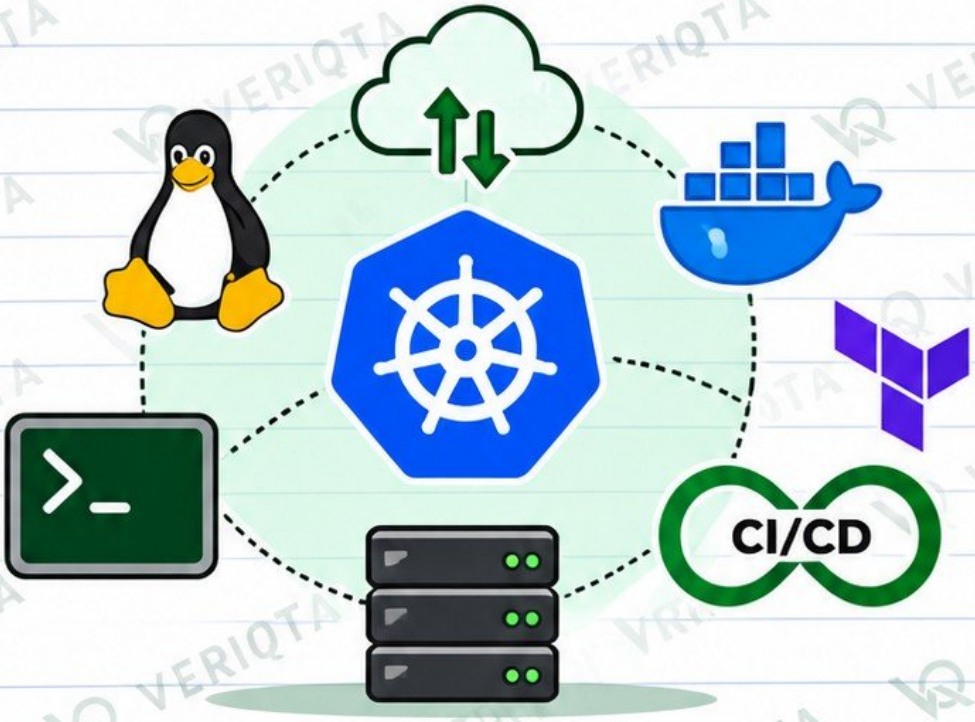
VERIQTA

PLATFORM ENGINEERING BOOTCAMP



@veriqta | @veriqta | @veriqta | @veriqta

@veriqta | @veriqta

WHAT IS PLATFORM ENGINEERING?

1. WHAT IS PLATFORM ENGINEERING?

Platform engineering is the practice of building and operating internal platforms that provide developers with self-service tools, templates, and infrastructure to build, deploy, and run applications quickly, safely, and consistently.

2. WHY DOES IT EXIST?

- Developers were slowed down by manual infrastructure requests.
- Different teams used inconsistent tools and configurations.
- Environments were hard to manage.
- Deployments were slow and error prone.
- Organizations needed a more secure, standardized, and scalable way to deliver infrastructure and tooling.

3. THE PROBLEM IT SOLVES

- Slow and manual environment setup
- Inconsistent configurations
- Duplicate work across teams
- Security and compliance gaps
- Difficult scaling and maintenance
- Developers spending time on infrastructure instead of building features

4. HOW IT WORKS (SIMPLE VIEW)



5. HOW IT DIFFERS FROM DEVOPS AND SRE

PLATFORM ENGINEERING

- Builds internal platforms for developers
- Focuses on self-service, standardization and automation
- Provides tools, templates and infrastructure
- Acts as a product for developers

DEVOPS

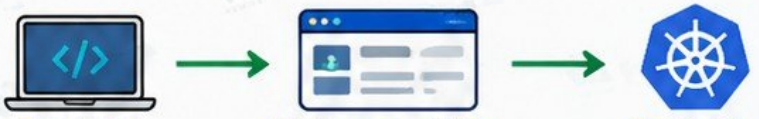
- Focuses on culture, collaboration and delivery
- Brings development and operations together
- Builds and improves CI/CD, automation and workflows
- Includes platform engineering as part of the broader DevOps practices

SITE RELIABILITY ENGINEERING (SRE)

- Focuses on reliability, availability and performance
- Uses monitoring, alerting and incident management
- Works on SLis, SLOs and error budgets
- Often uses the platform built by the platform engineering team

6. REAL WORLD EXAMPLE

A developer needs a new application environment. Instead of opening multiple tickets, they go to the internal platform, select a template, and the platform automatically provisions a secure Kubernetes environment with all required tools, monitoring, and policies.



7. KEY TAKEAWAY

Platform engineering gives developers a paved road to build and deploy applications, while the platform team manages the complexity, security and infrastructure behind the scenes.

HOW AN INTERNAL DEVELOPER PLATFORM WORKS

Explore Internal Developer Platforms (IDPs), developers as platform customers, self-service workflows, golden paths, and the platform team.

1. WHAT IS AN INTERNAL DEVELOPER PLATFORM?

- A set of tools, templates, services and automation that help developers build, deploy and run applications without managing underlying infrastructure.
- Provides a self-service experience through a central platform (portal, CLI, or API).
- Standardizes and automates common tasks.
- Built and maintained by a dedicated platform team.

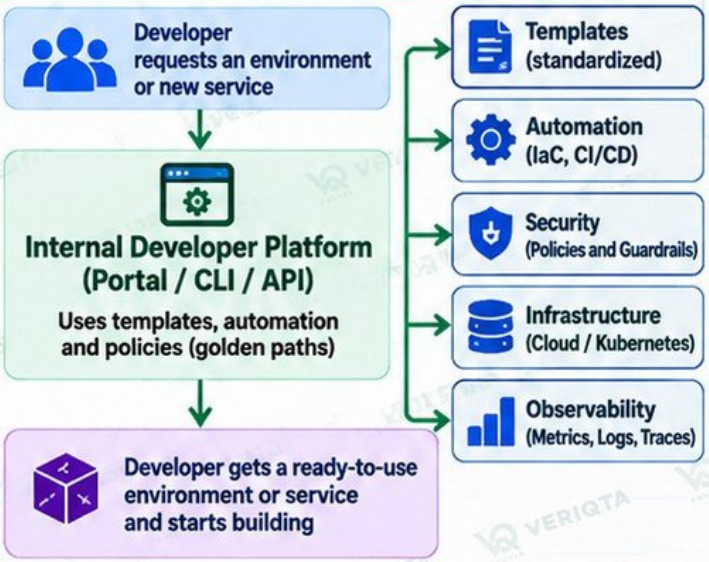
2. WHY DOES IT EXIST?

- Developers were slowed down by manual infrastructure requests.
- Inconsistent tools and configurations across teams.
- Longer time to deploy applications.
- Hard to maintain security and compliance.
- Organizations needed a faster, more standardized and secure way to deliver environments and tooling.

3. KEY PLAYERS

DEVELOPERS (PLATFORM CUSTOMERS)	• Use the platform to create and manage applications. • Focus on building features, not infrastructure.
PLATFORM TEAM	• Builds and maintains the platform. • Creates templates, automation and golden paths. • Ensures security, reliability and developer experience.
OPERATIONS / SRE	• Supports the underlying infrastructure. • Works with the platform team on reliability and observability.
SECURITY / COMPLIANCE	• Sets policies and guardrails. • Ensures secure and compliant environments.

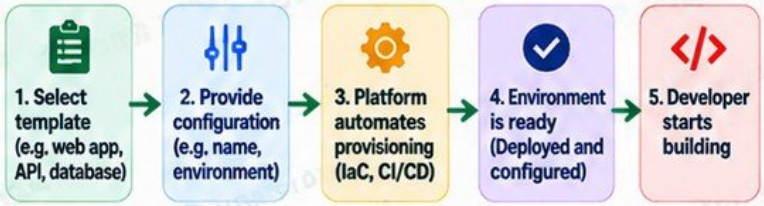
4. HOW IT WORKS (HIGH-LEVEL FLOW)



5. GOLDEN PATHS

- Pre-approved, best-practice templates and workflows for common use cases.
- Help developers build and deploy applications the right way.
- Include security, monitoring, and standard configurations.
- Reduce the need for custom, one-off solutions.

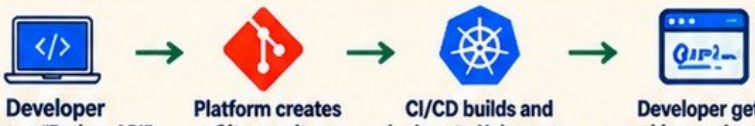
6. SELF-SERVICE WORKFLOWS (EXAMPLE)



7. BENEFITS

- Faster development and deployment.
- Consistent and standardized environments.
- Improved security and compliance.
- Better developer experience.
- Reduced operational overhead.
- Easier scaling across teams and projects.

8. REAL WORLD EXAMPLE



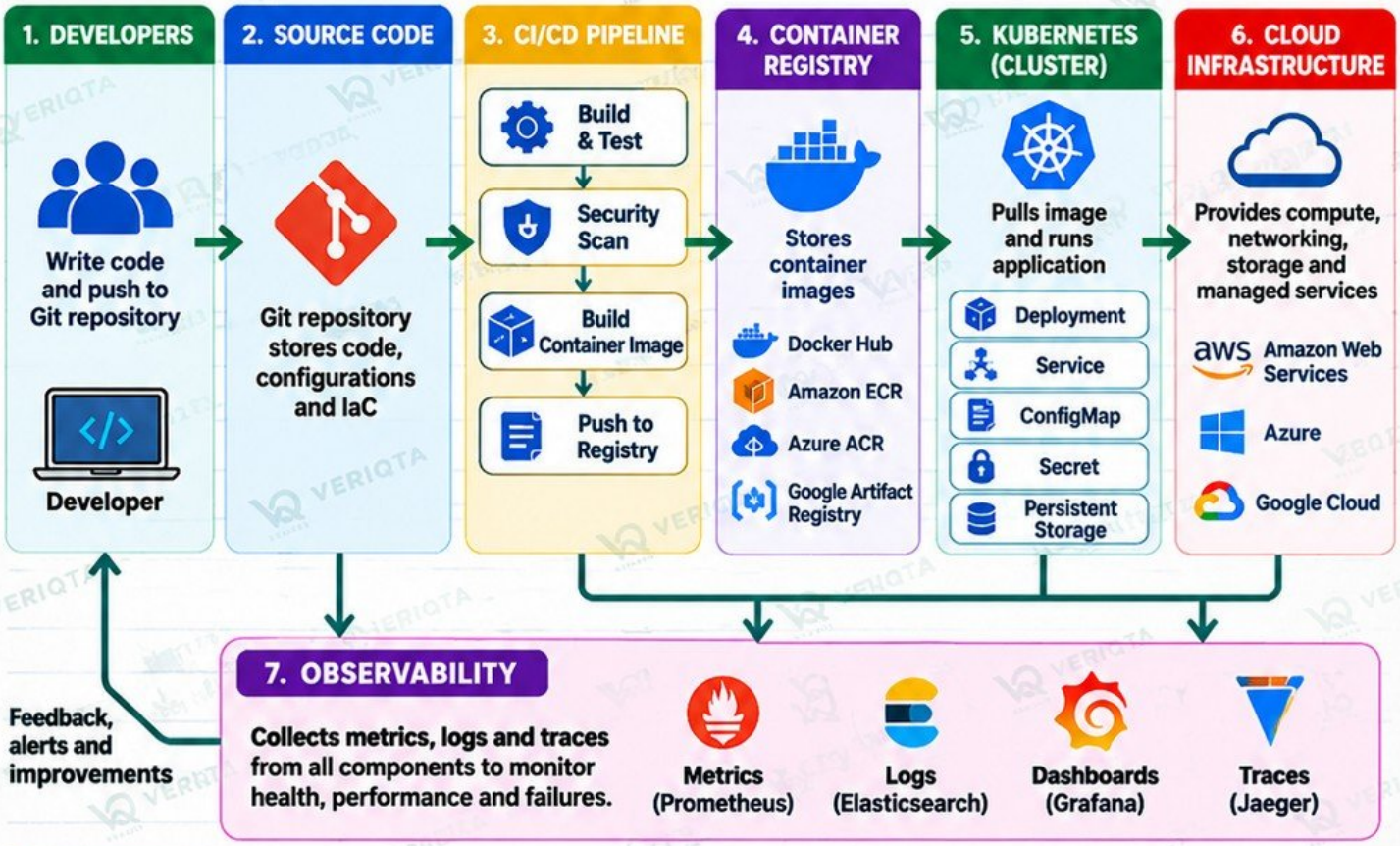
KEY TAKEAWAY

An Internal Developer Platform gives developers a secure, standardized and self-service way to build and run applications, while the platform team handles the complexity behind the scenes.

PLATFORM ENGINEERING ARCHITECTURE

VERIQTA

Follow an application from source code through CI/CD, container registries, Kubernetes, cloud infrastructure, and observability. Understand the platform's major components.



1. DEVELOPERS

- Write application code.
- Push to a Git repository.
- Use platform tools and templates for faster development.

2. SOURCE CODE REPOSITORY

- Stores application code, infrastructure code and configuration files.
- Triggers CI/CD pipelines when code changes.

3. CI/CD PIPELINE

- Builds and tests the application.
- Runs security scans.
- Builds a container image.
- Pushes the image to a container registry.

4. CONTAINER REGISTRY

- Stores versioned container images.
- Examples: Docker Hub, Amazon ECR, Azure ACR, Google Artifact Registry.
- Provides images for Kubernetes to pull.

5. KUBERNETES CLUSTER

- Pulls the container image from the registry.
- Runs the application using Deployments.
- Exposes the application using Services.
- Uses ConfigMaps, Secrets and Persistent Storage.

6. CLOUD INFRASTRUCTURE

- Provides the underlying compute, networking, storage and managed services.
- Examples: AWS, Azure, Google Cloud.
- Runs Kubernetes as a managed service (e.g., EKS, AKS, GKE) or self-managed clusters.

7. OBSERVABILITY

- Collects metrics, logs and traces from applications, Kubernetes and infrastructure.
- Helps detect issues, troubleshoot, and improve reliability.
- Common tools: Prometheus, Grafana, Elasticsearch, Jaeger.

7. KEY TAKEAWAY

Platform engineering connects development, automation, infrastructure and observability to provide a reliable, self-service environment for developers.

LINUX, NETWORKING, AND GIT FOUNDATIONS

Understand the operating system, processes, permissions, DNS, ports, SSH, repositories, and version control that support platform operations.

1. LINUX BASICS

- Linux is a multi-user operating system.
- It manages hardware, files, processes, users and network connections.
- Platform engineers use Linux to run applications, containers, and infrastructure services.



USEFUL COMMANDS

`uname -a`	Show OS information
`df -h`	Check disk space
`ls -l`	List files with details
`cd /path`	Change directory
`pwd`	Show current directory

2. PROCESSES AND SERVICES

- A process is a running program.
- A service is a long-running process that starts automatically (e.g. nginx, sshd).
- Platform engineers check and manage processes and services to keep systems healthy.



USEFUL COMMANDS

`ps aux`	List running processes
`top`	Real-time process view
`systemctl status nginx`	Check service status
`systemctl start nginx`	Start a service
`systemctl enable nginx`	Start on boot

3. FILE PERMISSIONS

- Permissions control who can read, write or execute files and directories.
- Platform engineers secure configuration files, scripts and sensitive data.



USEFUL COMMANDS

`ls -l`	View permissions
`chmod 755 file`	Change permissions
`chown user:group file`	Change owner and group

4. NETWORKING BASICS

- Computers communicate using IP addresses, ports and protocols.
- Platform engineers use networking to connect services, troubleshoot issues and secure access.



USEFUL COMMANDS

`ip a`	Show IP addresses
`ip r`	Show routing table
`ping <host>`	Test connectivity
`curl http://<host>`	Test HTTP access
`ss -tuln`	Show listening ports

5. DNS (DOMAIN NAME SYSTEM)

- DNS converts domain names (e.g. example.com) to IP addresses.
- Platform engineers use DNS for service discovery, load balancers and internal applications.



USEFUL COMMANDS

`nslookup example.com`	Query DNS record
`dig example.com`	Detailed DNS lookup
`host example.com`	Simple DNS query

6. PORTS AND PROTOCOLS

- Ports allow multiple services on the same server.
- Common protocols: HTTP (80), HTTPS (443), SSH (22).
- Platform engineers check open ports and connectivity.



USEFUL COMMANDS

`ss -tuln`	Show open ports
`nc -zv <host> 80`	Test if a port is open
`telnet <host> 443`	Test connectivity (if available)
`curl https://<host>`	Test HTTPS service

7. SSH (SECURE SHELL)

- SSH provides secure remote access to servers.
- Platform engineers use SSH keys instead of passwords.
- It is essential for managing Linux servers and cloud instances.



USEFUL COMMANDS

`ssh user@server`	Connect to a server
`ssh -i key.pem user@server`	Use a private key
`ssh-keygen`	Generate a new key pair

8. GIT AND REPOSITORIES

- Git tracks changes in your code and configuration files.
- Repositories store the history of your work.
- Platform engineers use Git for infrastructure code, configuration and application source code.



USEFUL COMMANDS

`git init`	Initialize a repository
`git clone <url>`	Copy a repository
`git add .`	Stage changes
`git commit -m "message"`	Save changes
`git push origin main`	Push to remote repository

KEY TAKEAWAY Linux, networking and Git are the core building blocks for platform engineering. They help you manage systems, connect services, secure access and control your code.

DOCKER AND CONTAINER FUNDAMENTALS

Understand images, containers, Dockerfiles, registries, container networking, and why platforms use containers to package applications.

1. WHAT IS DOCKER?

- Docker packages an application and its dependencies into a container.
- Containers run the same way on any environment (laptop, server or cloud).
- Platform engineers use Docker to create consistent, portable and reproducible environments for applications.

2. IMAGE vs CONTAINER

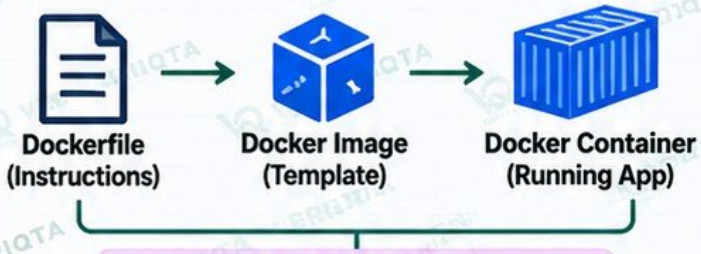
Docker Image

- Read-only template
- Contains the app, dependencies and config
- Used to create containers
- Stored in registries

Docker Container

- Running instance of an image
- Has a writable layer
- Runs the application
- Can be started, stopped, removed or recreated

3. DOCKER ARCHITECTURE (SIMPLE VIEW)



4. DOCKERFILE EXAMPLE

```
1  FROM ubuntu:22.04
2  WORKDIR /app
3  COPY . /app
4  RUN apt-get update && apt-get install -y python3
5  RUN pip3 install -r requirements.txt
6  EXPOSE  8000
7  CMD ["python3", "app.py"]
```

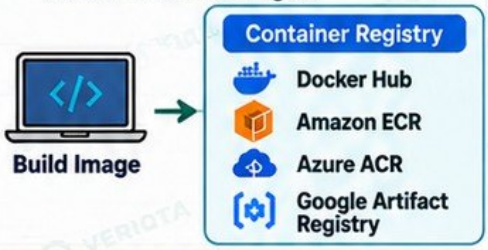
- Defines the base image, copies your code, installs dependencies and specifies how to run the application.
- Platform teams standardize Dockerfiles for consistency and security.

5. USEFUL DOCKER COMMANDS

COMMAND	WHAT IT DOES	EXAMPLE
`docker build -t app:v1 .`	Build an image	`docker build -t app:v1 .`
`docker images`	List local images	`docker images`
`docker run -d -p 8080:80 app:v1`	Run a container	`docker run -d -p 8080:80 app:v1`
`docker ps`	List running containers	`docker ps`
`docker logs <container>`	View container logs	`docker logs web`
`docker stop <container>`	Stop a container	`docker stop web`
`docker rm <container>`	Remove a container	`docker rm web`

6. DOCKER REGISTRY

- A registry stores Docker images.
- Examples: Docker Hub, Amazon ECR, Azure ACR, Google Artifact Registry.
- Platform engineers use registries to share and version images.



7. CONTAINER NETWORKING

- Containers can communicate with each other using Docker networks.
- Useful for multi-container applications (e.g. app + database).
- Docker provides a default bridge network and custom networks.



8. WHY PLATFORMS USE CONTAINERS

- Consistent environmnats across development, testing and production.
- Faster and more reliable deployments.
- Easy scaling and high availability.
- Resource isolation between applications.
- Supports microservices and cloud-native architectures.
- Works seamlessly with Kubernetes and modern CI/CD pipelines.

KEY TAKEAWAY

Docker packages your application and its dependencies into a portable, consistent and secure container, making it easier to build, deploy and manage applications across any environment.

KUBERNETES ARCHITECTURE FOR PLATFORM ENGINEERS

Learn about clusters, control planes, worker nodes, Pods, Deployments, Services, and how Kubernetes manages containerized workloads.

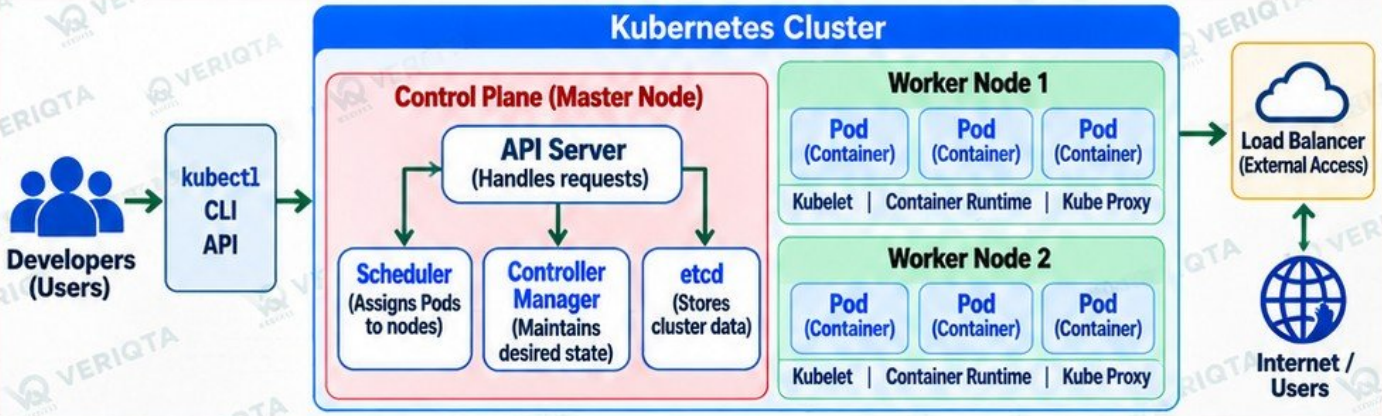
1. WHAT IS KUBERNETES?

- Kubernetes is an open-source container orchestration platform.
- It automatically deploys, scales, and manages containerized applications.
- It handles scheduling, networking, storage, health checks and self-healing.
- Platform engineers use Kubernetes to provide a reliable environment for applications.

2. KEY COMPONENTS

Cluster	A group of machines running Kubernetes.
Control Plane	Manages the cluster and makes decisions.
Worker Node	Runs your application workloads (Pods).
Pod	Smallest deployable unit (one or more containers).
Deployment	Manages Pod replicas, updates and rollbacks.
Service	Provides a stable way to access Pods.

3. KUBERNETES ARCHITECTURE (HIGH-LEVEL VIEW)



4. HOW IT WORKS (FLOW)

1. You create a Deployment (kubectl apply).
2. The API Server stores the desired state in etcd.
3. The Scheduler assigns Pods to worker nodes.
4. The Kubelet on each node runs the containers.
5. A Service provides a stable network endpoint.
6. Kubernetes monitors health and replaces failed Pods.

5. CORE KUBERNETES RESOURCES

Pod
- Runs containers
- Ephemeral
- Managed by Deployments

Deployment
- Manages Pods
- Handles updates
- Scaling and rollbacks

Service
- Stable access to Pods
- ClusterIP
- NodePort
- LoadBalancer

ConfigMap
- Stores configuration data
- Used by Pods

Secret
- Stores sensitive data
- Used by Pods

6. USEFUL KUBECTL COMMANDS

COMMAND	WHAT IT DOES
`kubectl get nodes`	List cluster nodes
`kubectl get pods -A`	List all Pods (all namespaces)
`kubectl get deployments`	List Deployments
`kubectl get services`	List Services
`kubectl describe pod <name>`	Show Pod details
`kubectl logs <pod>`	View Pod logs
`kubectl delete pod <name>`	Delete a Pod

7. TYPES OF SERVICES

TYPE	DESCRIPTION
ClusterIP	Internal access within the cluster (default).
NodePort	Exposes the service on each node's IP and port.
LoadBalancer	Provides an external load balancer (cloud).
ExternalName	Maps to an external DNS name.

8. WHERE PLATFORM ENGINEERS USE THIS

- Provide a stable environment for developer applications.
- Standardize deployments using templates and Helm charts.
- Implement self-service platforms and CI/CD pipelines.
- Manage scaling, updates, monitoring and security policies.
- Support multi-environment deployments (dev, staging, prod).

KEY TAKEAWAY

Kubernetes provides a reliable, scalable and automated platform to run containerized applications, and it is a core technology for modern platform engineering.

KUBERNETES APPLICATION CONFIGURATION

Understand namespaces, ConfigMaps, Secrets, resource requests and limits, health probes, and persistent storage.

1. NAMESPACES (ISOLATION)

- Namespaces allow you to isolate resources between teams, environments or applications.
- Useful for dev, staging, prod and multi-team setups.
- Resources (Pods, Services, ConfigMaps, etc.) exist inside a namespace.

EXAMPLE

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev
  labels:
    environment: dev
```

CREATE NAMESPACE

```
kubectl apply -f namespace.yaml

kubectl get namespaces
```

2. CONFIGMAP (NON-SENSITIVE CONFIGURATION)

- Stores non-sensitive configuration data as key-value pairs.
- Used for environment variables, application settings and configuration files.
- Keeps your application configuration separate from the image.

EXAMPLE

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
```

CREATE CONFIGMAP

```
kubectl apply -f configmap.yaml

kubectl get configmaps
```

3. SECRET (SENSITIVE DATA)

- Stores sensitive information like passwords, API keys and tokens.
- Data is base64 encoded (not encrypted by default).
- Use with environment variables or mounted files.

EXAMPLE

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD:bXlyYXNzd29yZA==
```

CREATE SECRET

```
kubectl apply -f secret.yaml

kubectl get secrets
```

4. RESOURCE REQUESTS AND LIMITS

- Requests guarantee the minimum resources for a container.
- Limits set the maximum resources a container can use.
- Helps with stability, performance and fair resource sharing between applications.

EXAMPLE (IN A POD)

```yaml
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

Request (minimum)	Limit (maximum)
250m CPU 256Mi Memory	500m CPU 512Mi Memory

5. HEALTH PROBES

- Liveness probe: checks if the container is running.
- Readiness probe: checks if the app is ready to receive traffic.
- Startup probe: gives extra time for slow-starting apps.

EXAMPLE (IN A POD)

```yaml
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
```

Liveness Probe — Restarts container if it fails.

Readiness Probe — Controls traffic to ready containers.

Startup Probe — Used for slow starting applications.

6. PERSISTENT STORAGE

- Persistent Volumes (PV) provide storage in the cluster.
- Persistent Volume Claims (PVC) allow applications to request storage.
- Used for databases, file storage and stateful applications.

Persistent Volume (PV) → Persistent Volume Claim (PVC) → Pod (Uses PVC)

EXAMPLE (PVC)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-pvc
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
```

CREATE PVC

```
kubectl apply -f pvc.yaml

kubectl get pvc
```

7. FULL EXAMPLE (DEPLOYMENT USING CONFIGMAP, SECRET, RESOURCES AND PROBES)

- This shows how multiple configurations come together in a real application.

```
kubectl apply -f deployment.yaml
kubectl get pods
kubectl describe pod <pod-name>
```

KEY TAKEAWAY

Application configuration in Kubernetes helps you build secure, reliable and scalable applications using namespaces, ConfigMaps, Secrets, resource limits, health probes and persistent storage.

HELM AND KUBERNETES APPLICATION PACKAGING

Learn Helm charts, templates, values files, releases, and how platform teams standardize application deployments.

1. WHAT IS HELM?

- Helm is a package manager for Kubernetes.
- It helps you package, configure and deploy applications using Helm charts.
- A Helm chart is a collection of templates, values and configuration files.
- It makes application deployment easier, repeatable and consistent across environments.

2. KEY CONCEPTS

Chart	A package of Kubernetes manifests.
Repository	A storage location for charts (public or private).
Template	YAML files with placeholders (Go templates).
Values File	Provides configuration values for the templates.
Release	A running instance of a chart in your cluster.

3. HELM CHART STRUCTURE

```
my-app/
├── Chart.yaml          # Chart metadata
├── values.yaml         # Default configuration
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   ├── secret.yaml
│   └── _helpers.tpl    # Reusable template helpers
└── charts/             # Dependencies (subcharts)
```

4. CHART.YAML (METADATA)

```yaml
apiVersion: v2
name: my-app
description: A sample application
type: application
version: 0.1.0
appVersion: "1.0.0"
```

Chart.yaml
Defines chart name, version and metadata

5. VALUES.YAML (CONFIGURATION)

```yaml
replicaCount: 2
image:
  repository: my-app
  tag: "1.0.0"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
```

values.yaml
Provides default values that can be overridden per environment (dev, staging, prod)

6. TEMPLATE EXAMPLE (DEPLOYMENT.YAML)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "my-app.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: my-app
        image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
        ports:
        - containerPort: 8080
```

templates/deployment.yaml
Uses Go templates to inject values from values.yaml

7. COMMON HELM COMMANDS

COMMAND	WHAT IT DOES
`helm create my-app`	Create a new chart
`helm lint .`	Check chart for errors
`helm template .`	Render templates locally
`helm install my-app .`	Install the chart
`helm upgrade my-app .`	Upgrade an existing release
`helm list`	List installed releases
`helm uninstall my-app`	Remove a release
`helm repo add <name> <url>`	Add a chart repository
`helm search repo <keyword>`	Search for charts
`helm dependency update`	Update chart dependencies

8. DEPLOYING AN APPLICATION (FLOW)

1. Prepare chart (chart + values)
2. Install or upgrade release (helm install/upgrade)
3. Kubernetes creates resources (Pods, Services, etc.)
4. Application runs in the cluster (verify and monitor)

9. CHART REPOSITORIES

- Public repositories: Artifact Hub, Bitnami, Kubernetes official charts.
- Private repositories: Used by platform teams for internal charts.
- Helps standardize and share approved application charts.

10. WHY PLATFORM TEAMS USE HELM

- Standardized and repeatable deployments.
- Easier environment-specific configuration.
- Supports reusable templates and best practices.
- Simplifies application lifecycle management.
- Works well with GitOps and CI/CD pipelines.

KEY TAKEAWAY

Helm makes it easy to package, configure and deploy Kubernetes applications consistently across environments, helping platform teams deliver reliable and standardized deployments.

CLOUD INFRASTRUCTURE FOR PLATFORM ENGINEERING

Explore cloud compute, virtual networks, subnets, load balancers, storage, identity, and the infrastructure supporting developer platforms.

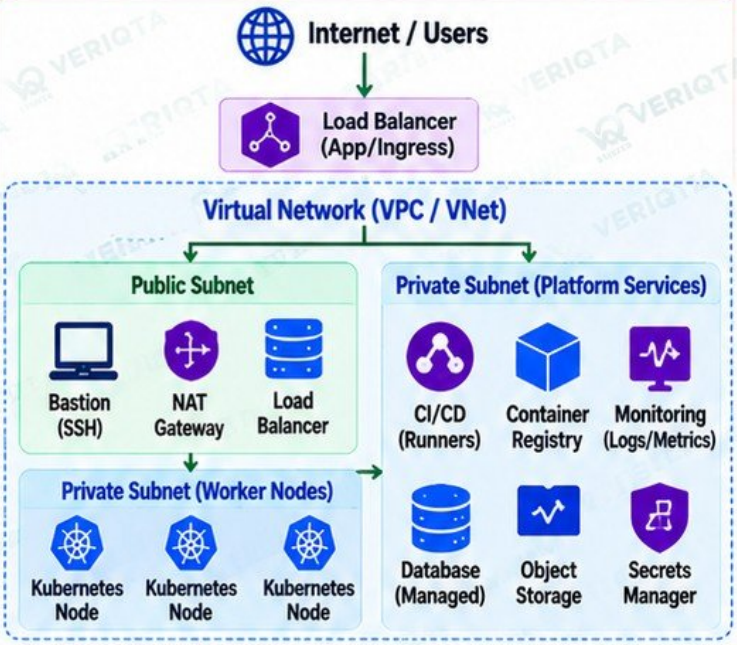
1. WHAT IS CLOUD INFRASTRUCTURE?

- Provides the compute, network, storage and security resources to run applications.
- Lets you build scalable, highly available and resilient platforms.
- Used by platform teams to host Kubernetes clusters, CI/CD, databases, monitoring and other platform services.
- Can be public cloud (AWS, Azure, GCP), hybrid or multi-cloud.

2. KEY CLOUD SERVICES FOR PLATFORMS

SERVICE	PURPOSE
Compute	Virtual machines, auto scaling, managed Kubernetes (EKS/AKS/GKE).
Networking	VPC/VNet, subnets, routing, load balancers.
Storage	Block, file and object storage for apps and data.
Identity & Access	Manage users, roles and permissions (IAM).
Security	Firewalls, security groups, encryption, secrets.
Observability	Metrics, logs and tracing for infrastructure and apps.

3. TYPICAL CLOUD ARCHITECTURE (HIGH-LEVEL VIEW)



4. COMPUTE OPTIONS

Virtual Machines (EC2 / Azure VM / GCE)

- Full control
- Good for custom workloads

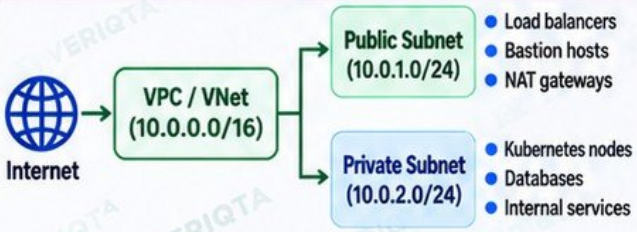
Managed Kubernetes (EKS / AKS / GKE)

- Managed control plane
- Scales automatically

Serverless (Lambda / Cloud Run)

- Run code without managing servers
- Good for event-driven tasks

5. VIRTUAL NETWORKS AND SUBNETS



6. LOAD BALANCERS

- Distribute traffic across multiple instances or Pods.
- Improve availability and fault tolerance.
- Types: Application Load Balancer (HTTP/HTTPS), Network Load Balancer (TCP/UDP).
- Used in front of Kubernetes clusters and applications.

7. STORAGE OPTIONS

Block Storage (Disks/Volumes)

- Attached to VMs
- Used for databases and stateful apps

File Storage (Shared)

- Shared across instances
- Used for shared files

Object Storage (S3 / Blob / GCS)

- Highly durable
- Used for artifacts, backups and logs

8. IDENTITY AND ACCESS MANAGEMENT (IAM)

- Manage users, roles and permissions.
- Use least privilege access.
- Service accounts for workloads (e.g. Kubernetes).
- Integrate with SSO for developer access.

9. SECURITY BEST PRACTICES

- Use private subnets for sensitive resources.
- Use security groups / network policies.
- Encrypt data at rest and in transit.
- Manage secrets with a secrets manager.
- Enable logging and monitoring.

10. WHY IT MATTERS FOR PLATFORMS

- Provides a reliable foundation for developer platforms.
- Enables scalable and secure deployments.
- Supports multi-environment setups (dev, staging, prod).
- Integrates with CI/CD, Kubernetes and observability tools.

KEY TAKEAWAY

Cloud infrastructure provides the compute, networking, storage, identity and security building blocks that power modern developer platforms and application workloads.

INFRASTRUCTURE AS CODE WITH TERRAFORM

Understand providers, resources, variables, state, modules, and the Terraform plan-and-apply workflow. Learn why state protection and review matter.

1. WHAT IS TERRAFORM?

- Terraform is an open-source Infrastructure as Code (IaC) tool.
- Lets you define, provision and manage infrastructure using code.
- Supports multiple cloud providers (AWS, Azure, GCP and more).
- Uses declarative configuration files.
- Helps create consistent, repeatable and version-controlled infrastructure.

Terraform

2. KEY CONCEPTS

Provider	Plugin that lets Terraform interact with a cloud platform.
Resource	A piece of infrastructure (e.g. VM, VPC, S3 bucket).
Variable	Input values to make configurations flexible.
State	Tracks real infrastructure and maps it to your code.
Module	Reusable and shareable Terraform code.
Plan	Shows what changes Terraform will make.
Apply	Creates or updates the infrastructure.

3. TERRAFORM WORKFLOW (PLAN AND APPLY)

1. Write Configuration (.tf files)
2. Initialize (Download providers)
3. Plan (Preview changes)
4. Apply (Create or update infrastructure)
5. Manage and update (Repeat)

4. TERRAFORM PROVIDERS

- Providers allow Terraform to manage resources in different platforms.
- Popular providers: AWS, Azure, Google Cloud, Kubernetes, Helm, GitHub, etc.
- Configure provider with credentials and region.

```
provider "aws" {
  region = "us-east-1"
  profile = "dev"
}
```

Supports multiple cloud platforms.

5. RESOURCE EXAMPLE (AWS EC2)

```
resource "aws_instance" "web" {
  ami           = "ami-0abc123"
  instance_type = "t3.micro"
  tags = {
    Name        = "web-server"
    Environment = "dev"
  }
}
```

Defines an EC2 instance with tags using Terraform.

6. VARIABLES (INPUT VALUES)

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

resource "aws_instance" "web" {
  ami           = "ami-0abc123"
  instance_type = var.instance_type
}
```

Variables make configurations flexible and reusable across environments.

7. MODULES (REUSABLE CODE)

```
modules/
└── vpc/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```

- Modules help you reuse code.
- Organize infrastructure into logical components.
- Makes it easier to maintain and share.

8. STATE FILE (TRACKS REAL INFRASTRUCTURE)

- Terraform stores state in a file (terraform.tfstate).
- Keeps a map of real resources and your configuration.
- Critical for tracking changes, detecting drift and managing updates.
- Should be stored securely (e.g. remote backend like S3 + DynamoDB, Azure Storage, or GCS).

STATE PROTECTION

- Store state remotely, not locally.
- Enable locking to prevent concurrent changes.
- Restrict access using IAM/permissions.
- Treat state as sensitive data.

9. COMMON COMMANDS

COMMAND	PURPOSE
terraform init	Download providers and initialize the project.
terraform validate	Check configuration for errors.
terraform plan	Show what changes will be made.
terraform apply	Create or update infrastructure.
terraform destroy	Delete the infrastructure.
terraform fmt	Format configuration files.
terraform state list	List resources in state.

10. WHY STATE AND REVIEW MATTER?

- State keeps Terraform and real infrastructure in sync.
- Incorrect state can cause unintended changes or resource loss.
- Reviewing the plan helps you catch mistakes before applying.
- Use pull requests and code reviews for all infrastructure changes.
- Protect state with proper access controls and locking.
- Ensures safer, more reliable and auditable deployments.

KEY TAKEAWAY Terraform lets you define, review and provision infrastructure as code. Use providers, resources, variables, modules and a secure state backend to build reliable and repeatable infrastructure.

CI/CD PIPELINES AND DEVELOPER WORKFLOWS

Follow a code change through build, test, security scanning, artifact publishing, deployment, verification, and rollback.

1. WHAT IS CI/CD?

- CI/CD automates the process of building, testing and deploying applications.
- CI (Continuous Integration): automatically builds and tests code changes.
- CD (Continuous Deployment/Delivery): releases changes to environments.
- Helps teams deliver software faster, safer and more reliably.

CI/CD

2. KEY BENEFITS

- Faster and more frequent releases.
- Early detection of issues.
- Consistent and repeatable deployments.
- Improved security and quality.
- Reduced manual work and human error.
- Supports multiple environments (dev, staging, prod).

3. CI/CD PIPELINE FLOW (HIGH-LEVEL VIEW)

1. Code Change (Push to Git)
2. Build (Compile & Package)
3. Test (Unit, Integration)
4. Security Scanning (Dependencies, Containers)
5. Publish Artifact (Container Image)
6. Deploy (to Environment)
7. Verify (Health Checks, Tests, Monitoring)
8. Rollback (if needed)

4. DEVELOPER WORKFLOW

1. Write Code
2. Push to Repository (Git)
3. CI/CD Pipeline Runs
4. Application Deployed to Environment
5. Developers Verify and Use

5. CI/CD TOOLS

TOOL	USE CASE
GitHub Actions	CI/CD pipelines in GitHub
GitLab CI/CD	Integrated CI/CD in GitLab
Jenkins	Self-hosted CI/CD server
Argo CD	GitOps for Kubernetes deployments
Tekton	Kubernetes-native CI/CD
CircleCI	Cloud-based CI/CD
AWS CodePipeline	CI/CD on AWS
Azure DevOps	CI/CD on Azure

6. EXAMPLE PIPELINE (GITHUB ACTIONS)

```yaml
name: CI CD Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Docker
        uses: docker/setup-buildx-action@v3
      - name: Build and push image
        run: |
          docker build -t my-app:${{ github.sha }} .
          docker push myregistry/my-app:${{ github.sha }}
```

GitHub Actions

7. CONTAINER IMAGE PUBLISHING

- Builds a container image from your application.
- Pushes the image to a container registry.
- Common registries: Docker Hub, AWS ECR, Google Artifact Registry, Azure ACR.

Build Image (docker build) → Push to Registry (docker push) → Docker Hub, AWS ECR, Google Artifact Registry, Azure Container Registry

8. DEPLOYMENT TO KUBERNETES (EXAMPLE)

```
# Deploy the new image to Kubernetes
kubectl set image deployment/my-app \
    my-app=myregistry/my-app:${GITHUB_SHA}
kubectl rollout status deployment/my-app
```

Kubernetes

9. VERIFICATION

- Check rollout status.
- Run health checks.
- Run automated tests.
- Monitor logs and metrics.
- Verify application functionality.

10. ROLLBACK (IF NEEDED)

- Revert to previous version using deployment history.
- Automated rollback on failure.
- Keep previous images and manifests.
- Validate the rollback was successful.

KEY TAKEAWAY CI/CD pipelines automate the journey from code to production, making software delivery faster, safer and more reliable with build, test, security, deployment and verification.

GITOPS WITH ARGO CD

argo CD

Understand desired state, Git as the configuration source, reconciliation, drift detection, and Kubernetes application synchronization.

1. WHAT IS GITOPS?

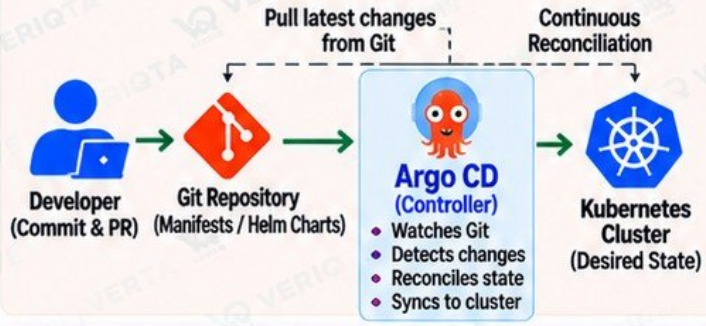
- GitOps uses Git as the single source of truth for desired state.
- Changes are made through pull requests.
- A controller (Argo CD) continuously reconciles the cluster to match the desired state in Git.
- Improves security, auditability and consistency.
- Widely used by platform teams for Kubernetes deployments.

GitOps

2. KEY CONCEPTS

Desired State	The state defined in Git (YAML manifests or Helm charts).
Source of Truth	Git repository (e.g. GitHub, GitLab, Bitbucket).
Reconciliation	Argo CD continuously checks and applies changes.
Drift Detection	Detects differences between the live cluster and Git.
Synchronization	Brings the cluster back to the desired state.
Declarative	Define what you want, not how to do it.
Automated	Sync automatically or manually (your choice).

3. HOW ARGO CD WORKS (ARCHITECTURE)



4. ARGO CD FEATURES

- Automated and manual synchronization.
- Drift detection and self-healing.
- Supports Helm charts, Kustomize and plain YAML.
- Web UI for easy management.
- Role-based access control (RBAC).
- Multi-cluster and multi-environment support.
- Application health and status monitoring.
- Integration with SSO, notifications and CI pipelines.

Argo CD Web UI

5. EXAMPLE APPLICATION (YAML)

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/veriqta/my-app.git
    targetRevision: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

This tells Argo CD to deploy the application from a Git repository to the Kubernetes cluster and keep it in sync.

6. SYNCHRONIZATION FLOW

1. Change pushed to Git (e.g. new image tag or config)
2. Argo CD detects the change (continuous reconciliation)
3. Argo CD compares desired vs live state (drift detection)
4. Argo CD syncs the cluster to match Git (apply changes)

7. SYNC POLICIES

POLICY	DESCRIPTION
Manual Sync	You click sync in the UI or run a command.
Automated Sync	Argo CD automatically applies changes.
Prune	Deletes resources that are no longer in Git.
Self-Heal	Fixes drift if someone manually changes the cluster.
Allow Empty	Allows deleting all resources (use with caution).

8. USEFUL COMMANDS

COMMAND	PURPOSE
argocd login <server>	Login to Argo CD server
argocd app list	List applications
argocd app get <name>	Get application details
argocd app sync <name>	Manually sync an application
argocd app delete <name>	Delete an application

9. DRIFT DETECTION EXAMPLE

LIVE CLUSTER (DRIFTED)		GIT (DESIRED STATE)
nginx:1.20	≠	nginx:1.21

Argo CD detects the difference and syncs the cluster to the desired state (nginx:1.21).

10. WHY GITOPS WITH ARGO CD?

- Keeps environments consistent.
- Detects and fixes configuration drift.
- Provides audit trail and change history.
- Reduces manual work and human error.
- Easier rollbacks through Git history.
- Scales across multiple clusters and environments.

KEY TAKEAWAY Argo CD uses Git as the source of truth to continuously reconcile and synchronize Kubernetes applications, ensuring your cluster always matches the desired state in Git.

DEVELOPER PORTALS AND BACKSTAGE

Backstage

Explore service catalogs, ownership metadata, documentation, software templates, and how Backstage helps developers discover and create services.

1. WHAT IS A DEVELOPER PORTAL?

- A single place for developers to discover services, documentation and tools.
- Provides self-service workflows to create and manage software.
- Improves consistency, productivity and governance.
- Backstage is a popular open-source developer portal built by Spotify.
- Integrates with your existing tools (Kubernetes, Git, CI/CD, monitoring, etc).

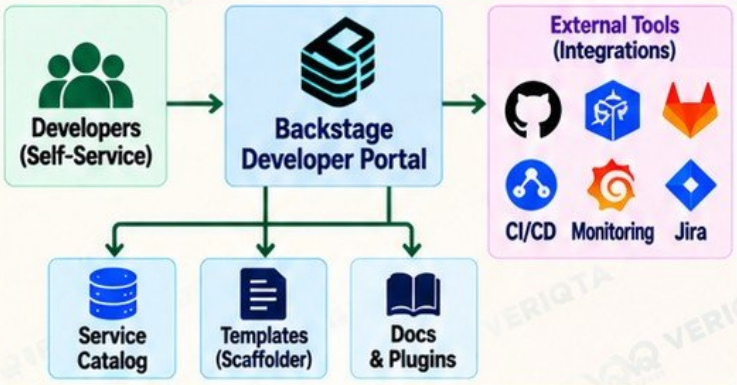


Developer Portal

2. KEY CONCEPTS

Service Catalog	Central list of software components (services, APIs, websites, libraries).
Ownership Metadata	Information about the service owner, team, system, lifecycle and contacts.
Documentation	Easy access to docs, runbooks and architecture info.
Software Templates	Reusable templates to create new services (e.g. Kubernetes, microservice, infrastructure).
Plugins	Integrations with tools (GitHub, GitLab, Argo CD, Grafana, Kubernetes, Jira, etc).
Scaffolder	Helps developers create new projects from templates.

3. BACKSTAGE ARCHITECTURE (HIGH-LEVEL)



4. SERVICE CATALOG EXAMPLE (YAML)

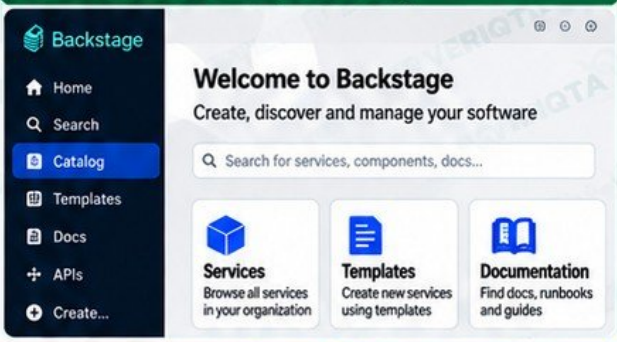
```yaml
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: payment-service
  description: Handles customer payments
  tags:
    - backend
    - critical
  annotations:
    github.com/project-slug: veriqta/payment-service
    backstage.io/techdocs-ref: dir:.
spec:
  type: service
  lifecycle: production
  owner: group:platform-team
  system: ecommerce
```

Defines a service in the Backstage catalog with ownership, tags and links to documentation and source code.

5. SOFTWARE TEMPLATES (SCAFFOLDER)



6. BACKSTAGE UI (EXAMPLE)



7. COMMON PLUGINS AND INTEGRATIONS



8. BENEFITS FOR DEVELOPERS

- Find existing services easily.
- Create new services quickly.
- Access documentation and runbooks.
- Understand ownership and contact points.
- Reduce boilerplate and repetitive work.
- Follow platform standards and best practices.
- Integrate with familiar tools.

9. WHY IT MATTERS

- Improves developer productivity.
- Enables self-service at scale.
- Increases consistency across services.
- Better visibility and ownership of software.
- Supports platform governance.
- Reduces siloed knowledge.

KEY TAKEAWAY Backstage provides a central developer portal with a service catalog, templates and documentation, helping developers discover, create and manage services faster and more consistently.

GOLDEN PATHS AND SELF-SERVICE INFRASTRUCTURE

Learn how platform teams create approved application templates, reusable workflows, automated provisioning, and safe self-service experiences.

1. WHAT ARE GOLDEN PATHS?

- Golden paths are approved, supported ways to build, deploy and operate applications.
- They provide the best practices, security controls and automation by default.
- Help developers move fast without making risky decisions.
- Reduce complexity, standardize infrastructure and improve reliability.
- Part of a self-service experience built and maintained by the platform team.

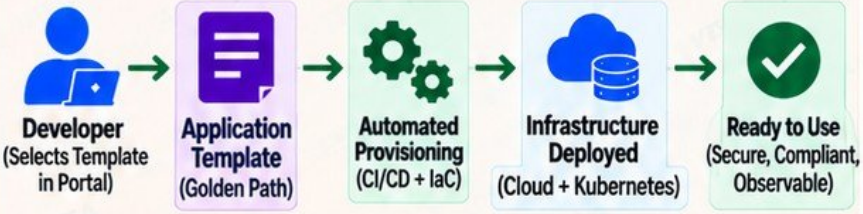


Golden Path

2. KEY BENEFITS

- **Faster delivery** — Developers can create new services quickly.
- **More secure** — Security and compliance built in by default.
- **Consistent and reliable** — Standardized and tested configurations.
- **Better developer experience** — Less time on setup, more time building features.
- **Lower operational overhead** — Fewer manual requests and repetitive work.

3. HOW SELF-SERVICE INFRASTRUCTURE WORKS



4. EXAMPLE GOLDEN PATHS

- New microservice (container + CI/CD).
- Data service (database + monitoring).
- Frontend application (static site + CDN).
- Serverless function (with IAM and monitoring).
- Batch job (scheduled workload).
- Complete environment (dev/staging/prod).

5. APPLICATION TEMPLATE EXAMPLE (BACKSTAGE)

Create a New Service

Service Name	user-service
Service Type	Node.js Microservice
Environment	Development
Repository	Create new repository

Create Service

What Happens Behind the Scenes?

1. Scaffolds code from template (e.g. Node.js, Python, Go).
2. Creates a Git repository.
3. Sets up CI/CD pipeline.
4. Provisions infrastructure with IaC (Terraform).
5. Applies Kubernetes manifests.
6. Configures monitoring, logging and security.

Reusable and standard application template.

6. TEMPLATE STRUCTURE (EXAMPLE)

```
service-template/
├── .github/
│   └── workflows/
├── src/
├── k8s/
├── terraform/
├── README.md
├── catalog-info.yaml
└── template.yaml
```

- Application code and dependencies.
- Kubernetes manifests for deployment.
- Terraform for infrastructure provisioning.
- Backstage template configuration.

7. SELF-SERVICE FLOW (END-TO-END)



8. PLATFORM TEAM RESPONSIBILITIES

- Create and maintain approved templates.
- Define standards, best practices and security controls.
- Automate infrastructure provisioning (IaC).
- Integrate with CI/CD, monitoring, logging and policy.
- Provide documentation and guidelines.
- Continuously improve developer experience.

9. TOOLS COMMONLY USED

- **Backstage** (Developer Portal)
- **Terraform** (IaC)
- **Argo CD** (GitOps)
- **GitHub** (Templates + CI/CD)
- **Kubernetes** (Deployment)
- **Cloud Provider** (AWS/Azure/GCP)
- **Monitoring** (Prometheus/Grafana)

10. WHY IT MATTERS

- Empowers developers to move fast and safely.
- Reduces manual work for platform teams.
- Ensures consistency across environments.
- Improves security, compliance and reliability.
- Makes platform engineering scalable.
- Helps teams focus on building business value.

KEY TAKEAWAY — Golden paths and self-service infrastructure give developers a fast, secure and consistent way to build and deploy applications using approved templates, automation and best practices.

PLATFORM APIs, AUTOMATION, AND SERVICE PROVISIONING

Understand APIs, automation workflows, service requests, asynchronous provisioning, and how a developer request becomes a deployed resource.

1. WHAT ARE PLATFORM APIS?

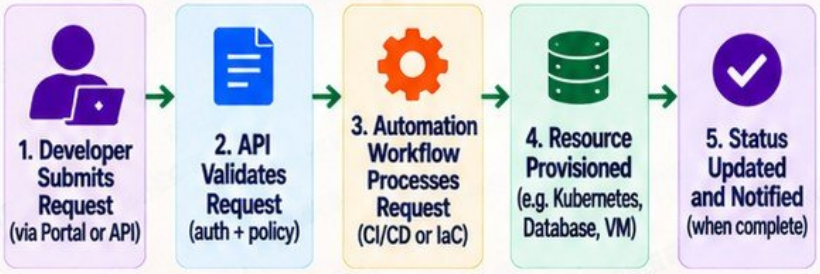
- APIs expose platform capabilities to developers.
- Allow programmatic access to create, configure and manage resources.
- Provide a consistent and secure way to interact with the platform.
- Enable self-service through automation.
- Used for services like environments, databases, Kubernetes, and more.



2. KEY CONCEPTS

API	Interface to access platform services.
Automation Workflow	Automates tasks based on a request.
Service Request	A developer's request for a resource.
Asynchronous Provisioning	Request is processed in the background.
Status Tracking	Check progress until completion.
Idempotency	Same request can be safely retried.
Events/Notifications	Inform users about progress and results.
Security	Authentication, authorization and audit logs.

3. HIGH-LEVEL FLOW (DEVELOPER REQUEST TO DEPLOYED RESOURCE)



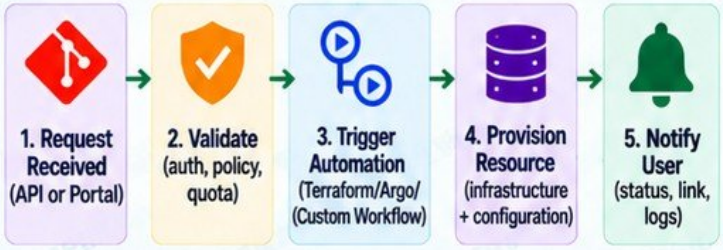
4. EXAMPLE API REQUEST (CREATE K&S NAMESPACE)

```
POST /api/v1/namespaces
Content-Type: application/json

{
  "name": "dev-team",
  "environment": "development",
  "owner": "team-a",
  "labels": {
    "cost-center": "1234",
    "team": "platform"
  }
}
```

A simple API request to create a Kubernetes namespace with metadata and labels.

5. AUTOMATION WORKFLOW (EXAMPLE)



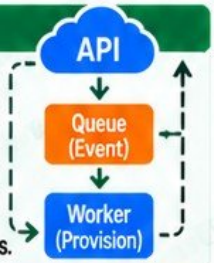
6. SAMPLE AUTOMATION (TERRAFORM + API)

```
# API triggers Terraform to create a namespace
resource "kubernetas_namespace" "example" {
  metadata {
    name   = var.namespace_name
    labels = {
      environment = var.environment
      owner       = var.owner
    }
  }
}
```

The platform API triggers a Terraform workflow to provision the resource.

7. ASYNCHRONOUS PROVISIONING

- Requests are processed in the background.
- API immediately returns a request ID.
- A worker (e.g. CI/CD, Terraform, custom controller) provisions the resource.
- Status can be checked via API or web UI.
- Events and notifications inform the user when it completes.



8. CHECK REQUEST STATUS (EXAMPLE)

```
GET /api/v1/requests/req-12345
Content-Type: application/json
{
  "id": "req-12345",
  "status": "in_progress",
  "resource": "namespace",
  "message": "Provisioning in progress",
  "created_at": "2026-10-11T10:00:00Z"
}
```

Use the request ID to check the provisioning status until it is completed.

9. REAL PLATFORM SERVICES EXAMPLES



10. BENEFITS

- Faster and consistent provisioning.
- Self-service for developers.
- Reduced manual work and errors.
- Better governance and security.
- Audit trail and visibility.
- Scales across teams and environments.

11. COMMON TOOLS

- Backstage (Service Catalog + APIs)
- Terraform (IaC automation)
- Argo CD / CI/CD (Automation)
- Kubernetes (Provisioned resources)
- GitHub/GitLab (Source and workflows)
- Vault (Secrets and credentials)

KEY TAKEAWAY Platform APIs and automation workflows allow developers to request and provision infrastructure and services safely, consistently and at scale, with full visibility and control.

PLATFORM SECURITY AND ACCESS CONTROL

Learn Role-Based Access Control (RBAC), least privilege, workload identity, secrets management, policy enforcement, and supply chain security.

1. WHY PLATFORM SECURITY MATTERS?

- Protects your infrastructure, applications and data.
- Prevents unauthorized access and accidental changes.
- Enables safe self-service for developers.
- Supports compliance and audit requirements.
- Reduces the impact of security incidents.
- Builds a secure and trusted platform for scale.

Secure Platform

2. KEY CONCEPTS

RBAC	Control who can do what in the cluster.
Least Privilege	Give only the minimum permissions needed.
Workload Identity	Secure access for applications and workloads.
Secrets Management	Store and use sensitive data securely.
Policy Enforcement	Use policies to enforce security rules.
Supply Chain Security	Keep your builds, artifacts and dependencies safe.
Audit and Monitoring	Track and detect suspicious activity.

3. RBAC IN KUBERNETES (HOW IT WORKS)

User (Developer) → Role (Defines Permissions) → RoleBinding (Grants Access to User/Group) → Kubernetes Resources (Pods, Deployments, Services, etc.)

EXAMPLE: ROLE AND ROLEBINDING

```
# Role: allow reading pods
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

```
# RoleBinding: grant to user
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: dev
subjects:
- kind: User
  name: developer1
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

4. WORKLOAD IDENTITY

- Allows applications to access cloud resources without hardcoded credentials.
- Uses service accounts, OIDC or cloud-native identity (e.g. IAM roles for service accounts).
- Improves security and reduces credential risk.

Pod (Service Account) → Workload Identity (OIDC/IAM) → Cloud Resources (e.g. AWS, Azure, GCP)

5. SECRETS MANAGEMENT

- Store sensitive data securely (passwords, tokens, API keys).
- Use Kubernetes Secrets or external tools (Vault, AWS Secrets Manager, Azure Key Vault).
- Avoid hardcoding secrets in code or container images.

Kubernetes Secrets · HashiCorp Vault · AWS Secrets Manager · Azure Key Vault

6. POLICY ENFORCEMENT

- Use policies to enforce security, compliance and best practices.
- Prevent risky configurations (e.g. privileged containers).
- Tools: Open Policy Agent (OPA), Kyverno, Gatekeeper.

OPA · Kyverno · Gatekeeper

7. SUPPLY CHAIN SECURITY

- Scan container images for vulnerabilities.
- Verify and sign images (e.g. Cosign).
- Use trusted base images and pin versions.
- Scan dependencies in your code.
- Prevent malicious or untrusted artifacts.

Source Code → Build → Scan (Vulnerabilities) → Sign (Cosign) → Deploy (Trusted Image)

8. ACCESS CONTROL BEST PRACTICES

- Use RBAC and least privilege.
- Separate environments (dev, staging, prod).
- Use workload identity for cloud access.
- Rotate secrets regularly.
- Enforce policies for security and compliance.
- Audit and monitor access and changes.
- Review permissions periodically.

9. COMMON SECURITY TOOLS

TOOL	USE CASE
Kubernetes RBAC	Access control in cluster
HashiCorp Vault	Secrets management
AWS Secrets Manager	Cloud secrets management
Azure Key Vault	Cloud secrets management
OPA	Policy enforcement
Kyverno	Kubernetes policy engine
Trivy	Image and filesystem scanning
Cosign	Image signing and verification
Snyk	Dependency and container scanning

10. REAL-WORLD PLATFORM SECURITY FLOW

Developer Request → Policy Check (OPA/Kyverno) → Get Credentials (Vault/Workload Identity) → Build & Scan (Cosign/Trivy) → Deploy to Cluster (Secure & Compliant)

11. KEY BENEFITS

- Stronger security and reduced risk.
- Controlled and auditable access.
- Safer self-service for developers.
- Protection against supply chain attacks.
- Compliance with industry standards.
- More reliable and trustworthy platform.

KEY TAKEAWAY A secure platform uses the right access controls, identity, secrets management, policy enforcement and supply chain security to give developers a safe and trusted self-service experience.

OBSERVABILITY FOR INTERNAL DEVELOPER PLATFORMS

Understand metrics, logs, traces, dashboards, alerting, and how platform teams investigate failures across applications and infrastructure.

1. WHAT IS OBSERVABILITY?

- Helps you understand what is happening inside your platform and applications.
- Uses metrics, logs and traces to give complete visibility.
- Helps detect, investigate and resolve issues quickly.
- Improves reliability, performance and developer experience.
- Essential for platform teams and self-service environments.

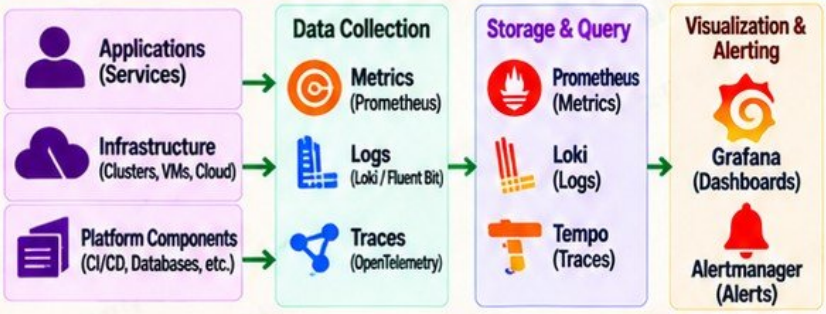


Observability

2. KEY CONCEPTS

Metrics	Numeric data (e.g. CPU, latency, request count).
Logs	Detailed event information (e.g. errors, requests).
Traces	End-to-end view of a request across services.
Dashboards	Visualize key signals for quick understanding.
Alerting	Notify you when something is wrong.
Correlation	Link metrics, logs and traces to find the root cause.
SLIs/SLOs	Measure reliability and user experience.

3. OBSERVABILITY ARCHITECTURE (HIGH-LEVEL)



4. THE THREE PILLARS

METRICS

- Numeric data
- Time series
- Aggregated and queried
- Used for alerting and dashboards

Examples:
- CPU usage
- Memory usage
- Request count
- Error rate
- Latency (p95, p99)

LOGS

- Detailed events
- Text-based
- Include context and metadata
- Useful for troubleshooting

Examples:
- Application logs
- Error logs
- Access logs
- Audit logs
- System logs

TRACES

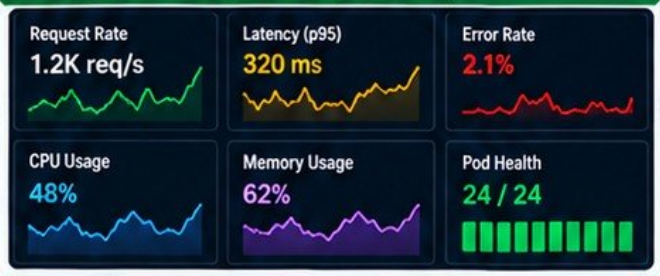
- Track a request across services
- Show latency each step
- Identify where failures occur
- Understand service dependencies

Examples:
- Request flow
- Service latency
- Error location
- Dependency calls
- End-to-end timeline

5. EXAMPLE: INVESTIGATING A FAILED REQUEST

1. User reports an error (500 response)
2. Check logs (find error message)
3. Check traces (find slow/failing service)
4. Check metrics (see spikes or high latency)
5. Identify root cause and fix

6. SAMPLE DASHBOARD (GRAFANA)



7. ALERTING EXAMPLE (PROMETHEUS + ALERTMANAGER)

```
groups:
- name: platform-alerts
  rules:
    - alert: HighErrorRate
      expr: rate(http_requests_total{status="5xx"}[5m]) > 0.05
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "High error rate detected"
        description: "Error rate is above 5% for 5 minutes."
```

Sends alert to Slack, Email or Teams when condition is met.

8. COMMON OBSERVABILITY TOOLS

Prometheus (Metrics) · Grafana (Dashboards) · Loki (Logs) · Tempo (Traces) · OpenTelemetry (Instrumentation) · Alertmanager (Alerting)

9. BEST PRACTICES

- Instrument applications with OpenTelemetry.
- Use meaningful logs and structured formats.
- Set SLIs, SLOs and error budgets.
- Create actionable alerts, not noisy alerts.
- Correlate metrics, logs and traces.
- Monitor platform components (CI/CD, clusters, etc.).
- Regularly review dashboards and alerts.

10. WHY IT MATTERS

- Faster incident detection and resolution.
- Better platform reliability.
- Improved developer experience.
- Reduces downtime and risk.
- Helps you understand system behavior at scale.
- Supports a secure and stable self-service platform.

KEY TAKEAWAY Observability gives platform teams the visibility they need to detect, investigate and fix issues quickly, ensuring a reliable and high-performing internal developer platform.

PLATFORM RELIABILITY AND TROUBLESHOOTING

Learn how to investigate failed deployments, unhealthy Pods, unavailable services, resource exhaustion, and configuration drift using evidence.

1. WHY RELIABILITY MATTERS?

- Keeps applications available and performing as expected.
- Reduces downtime and business impact.
- Helps you detect and fix issues quickly.
- Builds trust with developers and users.
- Improves the stability of the platform and infrastructure.
- Uses evidence and data, not assumptions.



2. COMMON PLATFORM ISSUES

ISSUE	EXAMPLE
Failed deployments	Image pull error, failed health checks, rollout stuck.
Unhealthy Pods	CrashLoopBackOff, OOMKilled, not ready.
Unavailable services	Service has no endpoints, DNS issues, network policy.
Resource exhaustion	High CPU, high memory, disk full, node pressure.
Configuration drift	Live environment different from desired state.
Platform component issues	CI/CD, cluster, storage, monitoring, DNS, ingress.

3. TROUBLESHOOTING WORKFLOW (STEP-BY-STEP)

1. **Identify the issue** (what is broken?)
2. **Gather evidence** (logs, metrics, describe, events)
3. **Analyze the root cause** (check config, resources, dependencies)
4. **Fix the issue** (apply change, redeploy, rollback)
5. **Verify and monitor** (confirm recovery and prevent recurrence)

4. USEFUL KUBERNETES COMMANDS

```
# Check pod status
kubectl get pods -n <namespace>
# Describe a pod (see events and errors)
kubectl describe pod <pod-name> -n <namespace>
# View pod logs
kubectl logs <pod-name> -n <namespace>
# Check previous container logs
kubectl logs <pod-name> -n <namespace> --previous
# Check service and endpoints
kubectl get svc,ep -n <namespace>
# Check events (recent issues)
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

5. EXAMPLE: FAILED DEPLOYMENT INVESTIGATION

1. **Deployment stuck** — Rollout not completing
2. **Check status** — kubectl rollout status deployment/app -n prod
3. **Check logs** — kubectl logs -pod-name- -n prod
4. **Find root cause** — Image pull error (wrong tag or missing secret)
5. **Fix and redeploy** — Update image tag or secret and redeploy

6. EXAMPLE: UNHEALTHY POD (CRASHLOOP)

1. **Pod crashing** — Pod in CrashLoopBackOff
2. **Check logs** — kubectl logs -pod-name- --previous
3. **Analyze** — Find error (e.g. missing config, DB not reachable)
4. **Fix** — Update config or dependency
5. **Verify** — Pod is running and healthy

7. EXAMPLE: UNAVAILABLE SERVICE

1. **Service not reachable** — Users cannot access the app
2. **Check service and endpoints** — kubectl get svc,ep -n prod
3. **Investigate** — No endpoints (pods not ready)
4. **Fix** — Fix pod issues, check selectors and readiness probes
5. **Confirm** — Endpoints exist and service is reachable

8. RESOURCE EXHAUSTION TROUBLESHOOTING

1. **Detect high usage** — High CPU, memory or disk
2. **Investigate** — kubectl top pods kubectl top nodes df -h
3. **Find cause** — Memory leak, large logs, high traffic, no limits
4. **Fix** — Set resource limits, optimize application
5. **Monitor** — Watch metrics to ensure stability

9. CONFIGURATION DRIFT INVESTIGATION

1. **Compare config** — Check live vs Git (GitOps/Argo CD)
2. **Check cluster** — kubectl get resources -o yaml
3. **Find drift** — Live settings different from repo
4. **Apply fix** — Sync from Git or update config
5. **Verify** — Configuration matches desired state

10. BEST PRACTICES

- Use logs, metrics and events to investigate.
- Follow a step-by-step troubleshooting process.
- Check recent changes (deployments, config).
- Validate dependencies (databases, APIs, DNS).
- Use monitoring and alerting for early detection.
- Document findings and fixes.
- Implement preventative measures.

11. COMMON TOOLS

kubectl	Kubernetes troubleshooting
Prometheus	Metrics and alerting
Grafana	Dashboards and visualization
Loki	Log aggregation
Jaeger	Distributed tracing
Argo CD	GitOps and sync status
kubectx/ stern	Log tailing and context switch

KEY TAKEAWAY — Platform reliability comes from systematically investigating issues using real evidence, understanding the full system, and applying fixes that prevent the same problems in the future.

PLATFORM OPERATIONS, GOVERNANCE, AND DEVELOPER EXPERIENCE

Explore platform upgrades, backup and recovery, cost awareness, service ownership, documentation, developer feedback, and meaningful platform performance measures.

1. WHAT DOES PLATFORM OPERATIONS COVER?

- Keeps the platform reliable, secure and up to date.
- Handles upgrades, patches and maintenance.
- Ensures backup and disaster recovery.
- Manages cost and resource usage.
- Defines service ownership and governance.
- Improves the developer experience (DX).
- Measures platform performance and value.



Operate
Govern
Improve

2. KEY AREAS AND GOALS

AREA	GOAL
Platform Upgrades	Keep platform components current and stable.
Backup and Recovery	Protect data and enable fast recovery.
Cost Awareness	Monitor and optimize spending.
Service Ownership	Clear ownership and accountability.
Documentation	Maintain accurate and useful docs.
Developer Feedback	Understand developer needs and pain points.
Performance Measures	Track metrics that show real platform value.

3. PLATFORM OPERATIONS LIFECYCLE

Plan (changes, upgrades, capacity) → **Implement** (upgrade, patch, configure) → **Validate** (test, monitor, verify) → **Operate** (monitor, support, optimize) → **Improve** (feedback, lessons learned, next steps)

4. PLATFORM UPGRADES (EXAMPLE PROCESS)

1. **Plan** — Review release notes, test in non-prod.
2. **Test** — Validate compatibility and functionality.
3. **Upgrade** — Apply changes using controlled rollout.
4. **Monitor** — Check health, logs and performance.
5. **Confirm** — Ensure stability and rollback plan ready.

5. BACKUP AND RECOVERY

1. **Backup** — Schedule regular backups (etcd, databases, configs).
2. **Store** — Keep backups in a separate secure location.
3. **Test Restore** — Regularly test recovery procedures.
4. **Document** — Keep clear recovery steps.

6. COST AWARENESS AND OPTIMIZATION

1. **Monitor** — Track resource usage and costs (by team/service).
2. **Analyze** — Find unused or underutilized resources.
3. **Optimize** — Right-size, set limits, use autoscaling.
4. **Set Budgets** — Define budgets and alerts.
5. **Review** — Regularly review spending with teams.

7. SERVICE OWNERSHIP AND GOVERNANCE

- **Define Owners** — Assign clear team ownership for each service.
- **Set Standards** — Define naming, labels, security and compliance.
- **Enforce Policies** — Use automated checks (OPA, Kyverno, etc.).
- **Track Changes** — Monitor changes and keep audit logs.

8. DEVELOPER EXPERIENCE (DX)

- **Self-Service** — Provide easy ways to create and manage services.
- **Good Documentation** — Clear guides, examples and FAQs.
- **Feedback Loops** — Collect and act on developer feedback.
- **Measure DX** — Track metrics like time to provision, success rate and satisfaction.

9. PLATFORM METRICS (EXAMPLES)

METRIC	WHAT IT MEASURES
Time to Provision	Time for a developer to get a new environment.
Deployment Success Rate	Percentage of successful deployments.
Platform Availability	Uptime of core platform services.
Mean Time to Recovery (MTTR)	Time to restore service after an issue.
Cost per Environment	Average cost of a dev/test/prod environment.
Developer Satisfaction	Feedback from developer surveys.
Adoption Rate	How many teams are using the platform.

10. COMMON TOOLS

- Kubernetes (operations)
- Helm (upgrades)
- Argo CD (management)
- Prometheus (monitoring)
- Grafana (dashboards)
- AWS (cost tools)
- OpenCost (cost visibility)
- Backstage (service catalog)

11. BEST PRACTICES

- Plan and test changes before production.
- Automate operations where possible.
- Maintain backups and test recovery regularly.
- Monitor cost and set budgets.
- Use clear ownership and documentation.
- Listen to developer feedback.
- Track meaningful metrics.
- Continuously improve based on data and real user needs.

KEY TAKEAWAY — A successful platform is reliable, well-governed, cost-aware, and easy to use, with clear ownership, good documentation and continuous improvement driven by real developer feedback and metrics.

BUILD YOUR FIRST INTERNAL DEVELOPER PLATFORM

Bring the bootcamp together through a guided platform engineering project using a local learning environment. You will build, deploy, automate and observe a real application from code to production.

1. PROJECT OVERVIEW

- Build a simple internal developer platform for a sample application.
- Use local tools to avoid unnecessary cloud charges.
- Learn the full flow: code → container → Kubernetes → Helm → CI → Git → Argo CD → observability → troubleshooting.
- Document the developer workflow and cleanup.

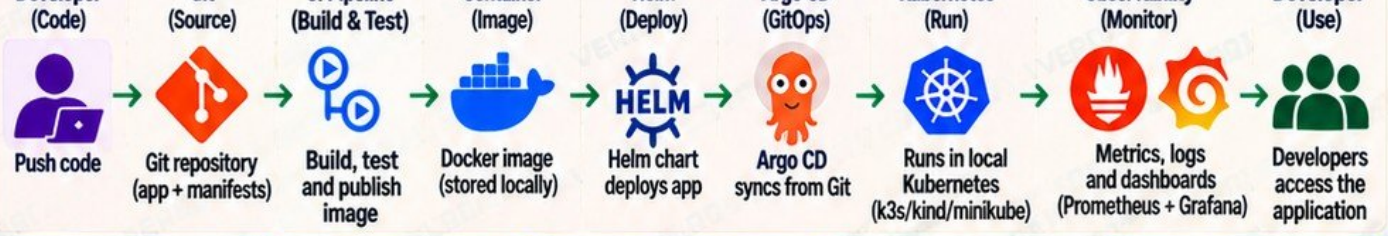


End-to-End Platform Project

2. LEARNING OBJECTIVES

1. Prepare a local Kubernetes environment.
2. Containerize a sample application.
3. Deploy it using Kubernetes and Helm.
4. Create a basic CI pipeline.
5. Store deployment configuration in Git.
6. Configure Argo CD for GitOps delivery.
7. Add health checks and basic observability.
8. Verify the deployment and troubleshoot a controlled failure.
9. Document the developer workflow and cleanup process.

3. HIGH-LEVEL ARCHITECTURE



4. LOCAL KUBERNETES SETUP

- Use kind, minikube or k3s.
- Create a local cluster.
- Verify nodes and pods.

```
# Using kind
kind create cluster
 --name platform
kubectl get nodes
kubectl get pods -A
```

5. CONTAINERIZE THE APPLICATION

- Use a simple app (e.g. Node.js or Python).
- Create a Dockerfile.
- Build and test the image.

```
docker build -t myapp:1.0 .
docker run -p 8080:8080
myapp:1.0
docker ps
```

6. DEPLOY WITH KUBERNETES & HELM

- Create a Helm chart.
- Deploy the application.
- Verify the deployment.

```
helm install myapp
./chart -n dev
 --create-namespace
kubectl get pods -n dev
kubectl get svc -n dev
```

7. BASIC CI PIPELINE

- Build and test the app.
- Build container image.
- Push to a local registry (or GitHub Container Registry).

```
# .github/workflows/ci.yml
jobs:
  build:
    steps:
      - run: docker build .
      - run: docker push ...
```

8. GITOPS WITH ARGO CD

- Store Kubernetes manifests and Helm chart in Git.
- Install and configure Argo CD.
- Sync the application.

```
argocd repo add <repo-url>
argocd app create myapp
argocd app sync myapp
argocd app get myapp
```

9. OBSERVABILITY AND HEALTH CHECKS

- Deploy Prometheus and Grafana (or use kube-prometheus).
- Add health checks to the application.
- Create dashboards and alerts.

```
kubectl apply -f monitoring/
kubectl get pods -n monitoring
# App health check
curl http://localhost:8080/health
```

10. TROUBLESHOOT A CONTROLLED FAILURE

- Break the application (e.g. wrong image tag).
- Use logs, events and metrics to find the issue.
- Fix and redeploy.

```
kubectl logs <pod> -n dev
kubectl describe pod <pod> -n dev
kubectl get events -n dev --sort-by=.lastTimestamp
helm upgrade myapp ./chart -n dev
```

11. DOCUMENT AND CLEANUP

- Document the architecture.
- Write a simple developer guide.
- Include troubleshooting steps.
- Clean up all resources.

```
helm uninstall myapp -n dev
kubectl delete namespace dev
kind delete cluster --name platform
```

12. TOOLING USED (LOCAL ENVIRONMENT)

kind / minikube k3s · Docker · Helm · Git · GitHub Actions (CI) · Argo CD (GitOps) · Prometheus (Metrics) · Grafana (Dashboards)

13. KEY OUTCOMES

- A working internal developer platform.
- Real end-to-end deployment workflow.
- GitOps-based delivery with Argo CD.
- Basic observability and health checks.
- Practical troubleshooting experience.
- Clear documentation and cleanup process.
- Reusable foundation for larger projects.

14. BEST PRACTICES

- Use local tools to avoid cloud costs.
- Keep configuration in Git.
- Automate builds and deployments.
- Use health checks and monitoring.
- Follow least privilege and security basics.
- Document everything.
- Clean up resources when finished.

KEY TAKEAWAY You have built a simple but complete internal developer platform, gaining practical experience with the key tools, workflows and concepts used in real platform engineering teams.